

Regulator Systems: Combined State Feedback and Observer

The separation principle: design the controller and observer separately.

Ogata, *Modern Control Engineering*, Ch. 10.

In this tutorial you will:

- combine a state-feedback gain K with an observer gain G ,
- verify the separation principle in the augmented eigenvalues, and
- read off the observer-based compensator's transfer function.

Step through with **Ctrl+Enter**, or render a report with **publish**.

When the full state is not measurable, the **separation principle** justifies designing the state-feedback gain K (from `PolePlacement.m`) and the observer gain G (from `StateObservers.m`) **independently**: the closed-loop system built from $u = -K\hat{x}$ with $\hat{x}$ supplied by the observer has characteristic polynomial equal to the **product** of the controller's and the observer's individually-designed polynomials. The combined controller-observer is called an **observer-based regulator** (or, with a reference input, a **compensator**).

Contents

- Plant
- Step 1: Design State-Feedback Gain (Controller Poles)
- Step 2: Design Observer Gain (Observer Poles, Faster)
- Step 3: Augmented Closed-Loop System
- Simulating the Regulator
- Before vs. After: Open-Loop Drift vs. Regulated Response
- Transfer Function of the Observer-Based Compensator
- Try it yourself

Plant

```
A = [0 1 0; 0 0 1; 0 -2 -3];
B = [0;0;1];
C = [1 0 0];
D = 0;
n = size(A,1);
```

Step 1: Design State-Feedback Gain (Controller Poles)

```
controller_poles = [-2+2j,-2-2j,-6];
K = acker(A,B,controller_poles);
fprintf('Controller gain K = '); disp(K)
```

```
Controller gain K =      48      30      7
```

Step 2: Design Observer Gain (Observer Poles, Faster)

```
observer_poles = [-10,-11,-12];
G = acker(A',C',observer_poles)';
fprintf('Observer gain G = '); disp(G)
```

```
Observer gain G =      30
                   270
                   450
```

Step 3: Augmented Closed-Loop System

State the combined dynamics in terms of $[x; e]$ where $e = x - \hat{x}$:

$$\dot{x} = Ax - BK\hat{x} = (A - BK)x + BKe$$

$$\dot{e} = (A - GC)e$$

This block-triangular structure is the separation principle made explicit: the $2n \times 2n$ system matrix

$$\begin{bmatrix} A - BK & BK \\ 0 & A - GC \end{bmatrix}$$

has eigenvalues equal to the union of $\text{eig}(A - BK)$ and $\text{eig}(A - GC)$.

```
A_aug = [A-B*K, B*K; zeros(n), A-G*C];
fprintf('\nCombined system eigenvalues:\n')
disp(eig(A_aug))
fprintf('Controller poles and observer poles (union):\n')
disp([controller_poles observer_poles])

Combined system eigenvalues:
Columns 1 through 4

-2.0000 - 2.0000i -2.0000 + 2.0000i -6.0000 + 0.0000i -12.0000 + 0.0000i

Columns 5 through 6

-11.0000 + 0.0000i -10.0000 + 0.0000i

Controller poles and observer poles (union):
Columns 1 through 4

-2.0000 + 2.0000i -2.0000 - 2.0000i -6.0000 + 0.0000i -10.0000 + 0.0000i

Columns 5 through 6

-11.0000 + 0.0000i -12.0000 + 0.0000i
```

Simulating the Regulator

Initial-condition (regulator) response: the plant starts away from equilibrium, the observer starts at zero (so $e(0) = x(0)$), and the combined system is driven to zero by output feedback through the estimated state.

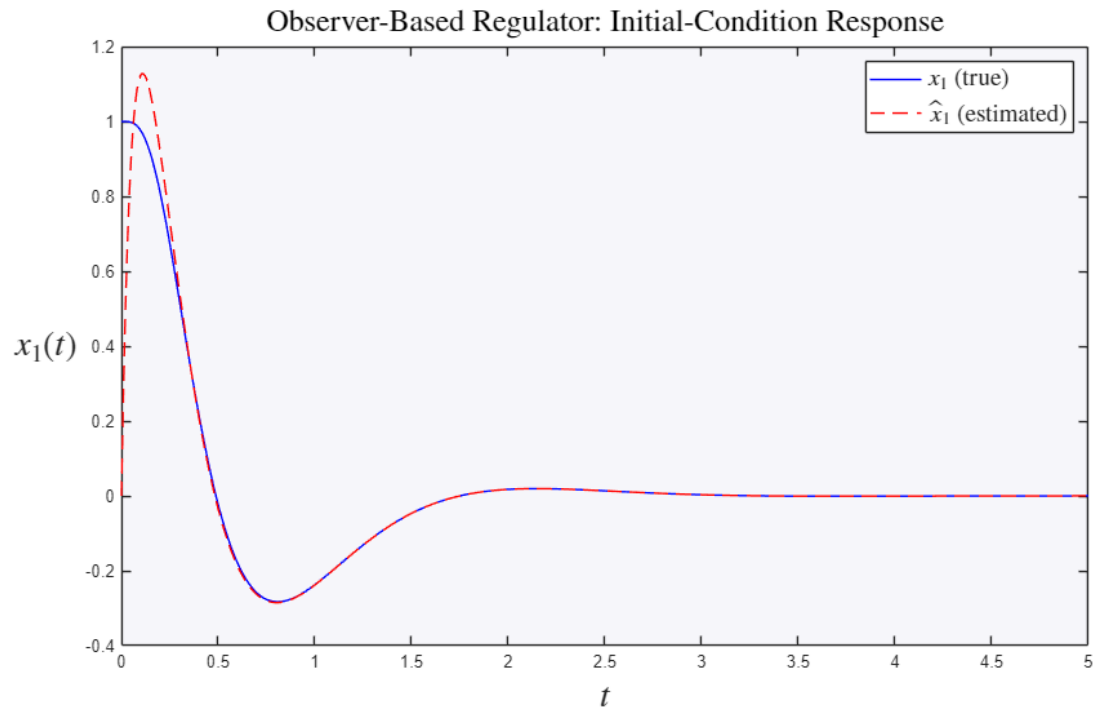
```
x0 = [1;0;0];
e0 = x0; % observer starts at zero estimate
z0 = [x0;e0];
C_aug = [eye(n) zeros(n); eye(n) -eye(n)]; % outputs: x and x_hat=x-e
sys_aug = ss(A_aug, zeros(2*n,1), C_aug, zeros(2*n,1));
t = 0:0.01:5;
[z,~] = initial(sys_aug,z0,t);
x_resp = z(:,1:n);
xhat_resp = z(:,n+1:end);

figure
plot(t,x_resp(:,1),'b',t,xhat_resp(:,1),'r--')
```

```

legend('$x_1$ (true)', '$\hat{x}_1$ (estimated)', 'Interpreter', 'latex', 'FontSize', 14)
title('Observer-Based Regulator: Initial-Condition Response', 'Interpreter', 'latex', 'FontSize', 18)
ylabel('$x_1(t)$', 'Interpreter', 'latex', 'FontSize', 20)
set(get(gca, 'YLabel'), 'Rotation', 0)
xlabel('$t$', 'Interpreter', 'latex', 'FontSize', 20)

```



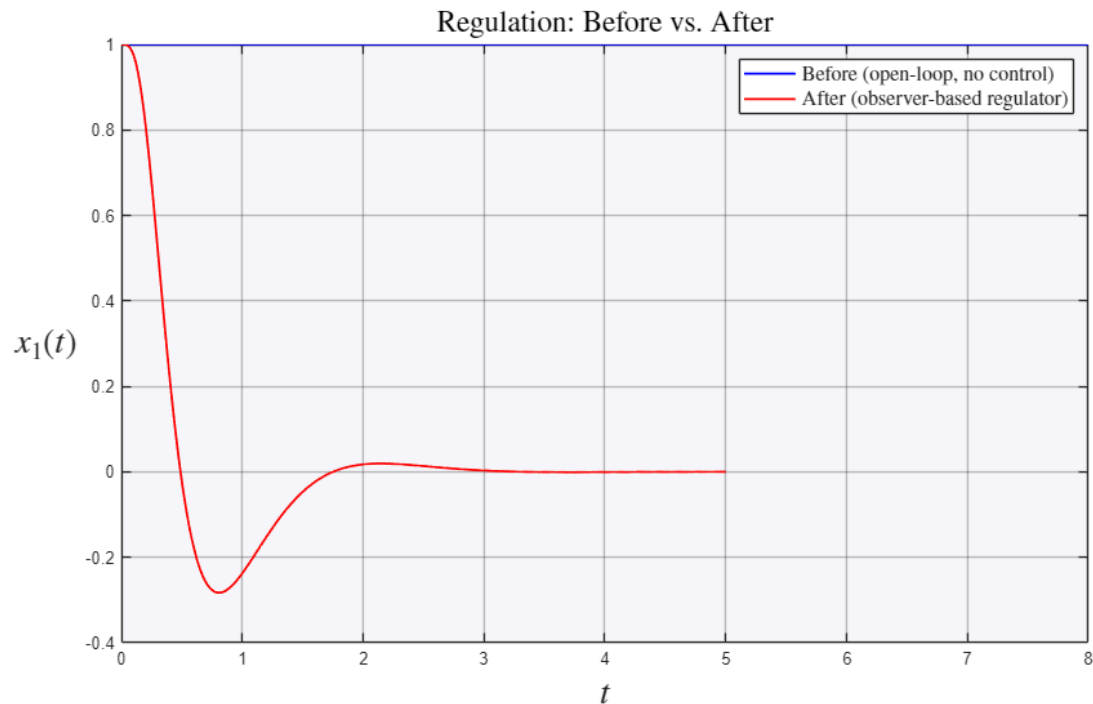
Before vs. After: Open-Loop Drift vs. Regulated Response

Without control the plant (a pole at the origin) never returns to zero from a disturbance; the observer-based regulator drives every state to zero using only the measured output y – and it does so even though the observer starts with no knowledge of the true state.

```

t2 = 0:0.01:8;
x_openloop = initial(ss(A,zeros(n,1),eye(n),0), x0, t2);
figure
plot(t2, x_openloop(:,1), 'b', 'LineWidth', 1.3)
hold on
plot(t, x_resp(:,1), 'r', 'LineWidth', 1.3)
hold off
grid on
legend('Before (open-loop, no control)', 'After (observer-based regulator)', ...
      'Interpreter', 'latex', 'FontSize', 12)
title('Regulation: Before vs. After', 'Interpreter', 'latex', 'FontSize', 17)
ylabel('$x_1(t)$', 'Interpreter', 'latex', 'FontSize', 20)
set(get(gca, 'YLabel'), 'Rotation', 0)
xlabel('$t$', 'Interpreter', 'latex', 'FontSize', 20)

```



Transfer Function of the Observer-Based Compensator

The dynamic output-feedback compensator that implements $u = -K\hat{x}$ internally (eliminating $\hat{x}$ as an explicit state visible to the plant) has its own transfer function $G_c(s)$ from $-y$ to u :

$$G_c(s) = K(sI - A + BK + GC)^{-1}G$$

```
A_c = A - B*K - G*C;
B_c = G;
C_c = -K;
D_c = 0;
Gc_compensator = tf(ss(A_c,B_c,C_c,D_c));
fprintf('\nObserver-based compensator transfer function:\n')
Gc_compensator
```

Observer-based compensator transfer function:

```
Gc_compensator =

-1.269e04 s^2 - 5.13e04 s - 6.336e04
-----
s^3 + 40 s^2 + 602 s + 4158
```

Continuous-time transfer function.

Try it yourself

- Change the observer poles and confirm the augmented eigenvalues stay exactly the controller poles union the observer poles (separation).
- Give the observer a worse initial guess and watch the estimation error decay independently of the regulation.